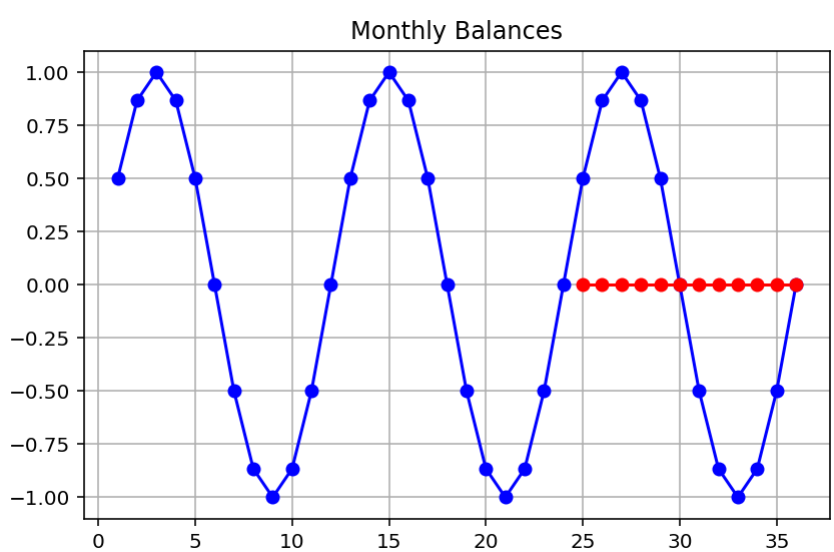
Time_Series_Forecast

Shows how selective reporting can make model seem to be better.

Also why MAPE is bad, and how to make stable WAPE error function.

In [1]:
```
import os, sys, math
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%config InlineBackend.figure_format = 'retina'
```

In [2]:
```
# suppose we have actual values for 3 years (36 months)
months = np.arange(1, 37, 1)
vals   = np.sin(months*2*math.pi/12.0)

# Suppose that we use first 24 months
# to train a simple forecasting model.
# It is so trivial - it predicts zero values.

months_pred = np.arange(25, 37, 1)
vals_pred   = np.full(12,0)         # 12 points with 0 values

fig, ax = plt.subplots(figsize=(6,4))
_ = ax.plot(months, vals, color="blue", marker='o')
_ = ax.plot(months_pred, vals_pred, color="red", marker='o')
_ = ax.set_title(f"Monthly Balances")
_ = ax.grid(True)

fig.tight_layout()
plt.show() ; # semicolon needed to remove text output on top
```



In [3]:
```
print(vals[24:].shape)
print(vals_pred.shape)
```

```
(12,)
(12,)
```

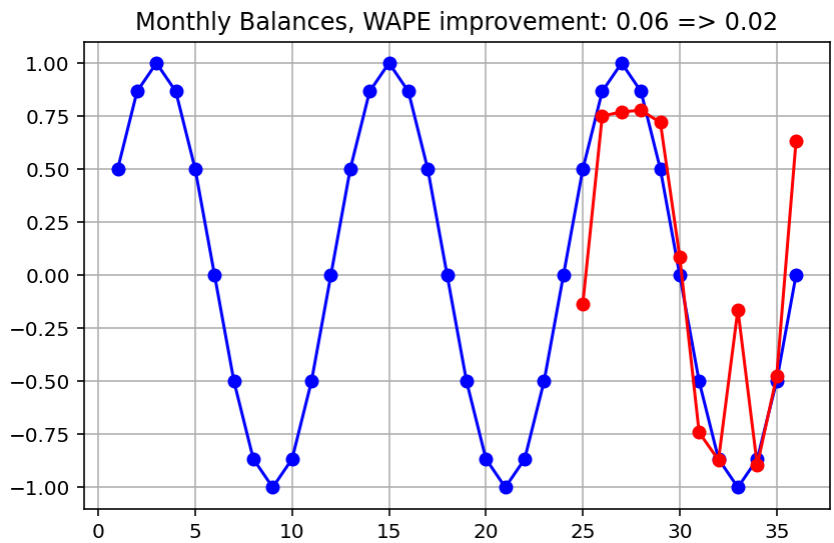
In [4]:
```
# Let's calculate the WAPE of the prediction
# https://www.baeldung.com/cs/mape-vs-wape-vs-wmape
# also see discussion at the bottom

# ----------------------------------------------------
def wape (actual, predicted, y_norm = None):
    """
    # calculate custom WAPE
    # (Weighted Absolute Percentage Error)
    # between two arrays (or lists)
    # y_norm (when provided) used as denominator
    """
    Na = len(actual)
    Np = len(predicted)
    # ----------------------------
    if Na == 0 or Np == 0 or Na != Np:
        return np.nan
    # ----------------------------
    # make sure we use numpy arrays
    np_act = np.array(actual)
    np_pre = np.array(predicted)
    # ----------------------------
    nomin = np.abs(np_act - np_pre).sum()
    # ----------------------------
    if y_norm and y_norm > 0:
        return min (1e4, nomin / (y_norm * Na))
    # ----------------------------
    v_max_act   = np.max(np_act)
    v_min_act   = np.min(np_act)
    v_amp_act   = v_max_act - v_min_act    # amplitude
    v_scale_act = np.abs(np_act).sum() / Na
    # ----------------------------
    if (v_scale_act == 0.0) and (nomin != 0.0):
        return 1e4  # this is arbitrary big number
    # ----------------------------
    if (v_amp_act <= 1e-4 * v_scale_act):
        shifted = (np_act - v_min_act) + v_scale_act
        denom = shifted.sum()
        if (denom == 0.0):
            return 1e4  # this is arbitrary big number
        else:
            return min (1e4, (nomin / denom))
    # ----------------------------
    # v_amp is reasonably large, shift in units of v_amp
    shifted = (np_act - v_min_act) + 5*v_amp_act
    denom = np.abs(shifted).sum()
    return min (1e4, (nomin / denom))

# ----------------------------------------------------
def mape (actual, predicted):
    """
    # calculate mape
    # (Mean Absolute Percentage Error)
    # between two arrays (or lists)
    """
    Na = len(actual)
```

```
    Np = len(predicted)
    if Na == 0 or Np == 0 or Na != Np:
        return np.nan
    actual_sum = np.absolute(actual).sum()
    if actual_sum <= 0:
        return np.nan
    mysum = 0
    for ii in range(Na):
        a = actual[ii]
        p = predicted[ii]
        c = a if np.abs(a) > 0.0001 else 0.0001
        v = np.abs((a-p)/c) / Na
        mysum += v
    return mysum

# ----------------------------------------------------
def myscale(x):
    """
    # find reasonable scale of positive variable
    """
    if x == 0:
        return 1e-6
    x = np.abs(x)
    n = 0
    if 0 < x < 1:
        while x < 1:
            n+=1
            x *=10
    elif x >= 10:
        while x >= 10:
            n-=1
            x /=10
    x = round(x+1)
    return x * ( 10 ** (-n) )
```

In [5]:
```
mytests = [
    ([0,0,0.0],[0,0,0]),
    ([0,0,0.001],[0,0,0]),
    ([0,0,0.000],[2000,2,2]),
    ([0,0,0.001],[2000,2,2]),
    ([0,0,0.001],[0,0,0.001]),
    ([10,10,10],[0,0,0.00]),
    ([10,10,10],[0,0,0.001]),
    ([10,10,10],[10,10,10]),
    ([10,10,10],[10.2,10.3,10.2]),

]
for tup in mytests:
    print(tup, f"{wape(tup[0],tup[1]):.4f}")
```

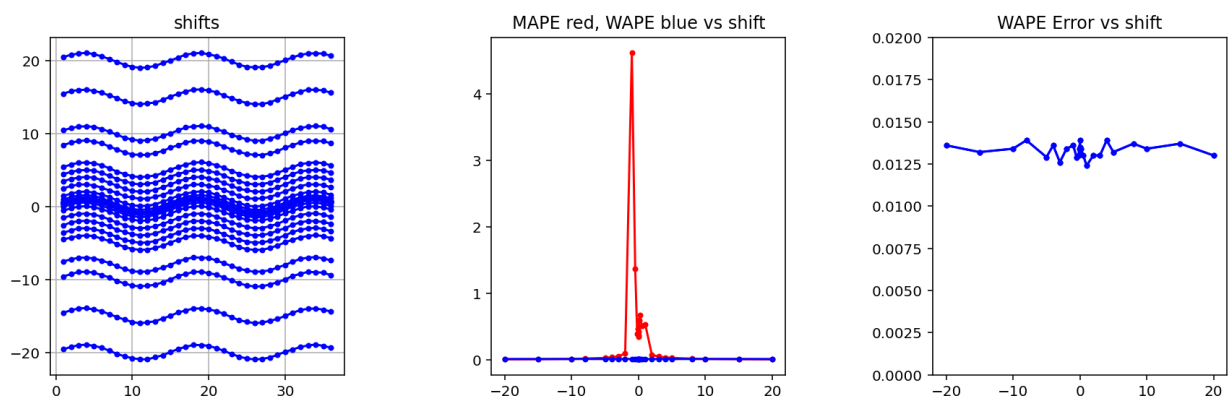
```
([0, 0, 0.0], [0, 0, 0]) 10000.0000
([0, 0, 0.001], [0, 0, 0]) 0.0625
([0, 0, 0.0], [2000, 2, 2]) 10000.0000
([0, 0, 0.001], [2000, 2, 2]) 10000.0000
([0, 0, 0.001], [0, 0, 0.001]) 0.0000
([10, 10, 10], [0, 0, 0.0]) 1.0000
([10, 10, 10], [0, 0, 0.001]) 1.0000
([10, 10, 10], [10, 10, 10]) 0.0000
([10, 10, 10], [10.2, 10.3, 10.2]) 0.0233
```

In [6]:
```
# Now let's try to improve prediction by using randomness

wape_best = wape(vals[24:], vals_pred)
wape_best_orig = wape_best
vals_best = np.full(12,0)

N = 1000
for counter in range(N):
    vals_temp = 1.8*(np.random.rand(12) - 0.5)
    wape_temp = wape(vals[24:], vals_temp)
    if wape_temp < wape_best:
        wape_best = wape_temp
        vals_best = vals_temp

# print(f"WAPE improvement: {wape_best_orig} => {wape_best}")
fig, ax = plt.subplots(figsize=(6,4))
_ = ax.plot(months, vals, color="blue", marker='o')
_ = ax.plot(months_pred, vals_best, color="red", marker='o')
_ = ax.set_title(f"Monthly Balances, "+f"WAPE improvement: {wape_best_orig:
_ = ax.grid(True)

fig.tight_layout()
plt.show() ; # semicolon needed to remove text output on top
```



Error Function Selection

In [7]:
```
months = np.arange(1, 37, 1)
vals   = np.sin(months*2*math.pi/15.0)
levels = [-20,-15,-10,-8,-5,-4,-3,
          -2,-1,-0.5,-0.2,-0.1,0.05,0,0.05,0.1,0.2,0.5,1,2,
          3,4,5,8,10,15,20]

fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(12,4))
ax0, ax1, ax2 = ax.flatten()

_ = ax0.set_title(f"shifts")
_ = ax1.set_title(f"MAPE red, WAPE blue vs shift")
_ = ax2.set_title(f"WAPE Error vs shift")
_ = ax0.grid(True)

errs_mape = []
errs_wape = []
for level in levels:
    rand_arr = 0.1*np.random.rand(len(vals))
    vals1 = vals+level+rand_arr
    vals2 = vals+level+0.2 # second ("predicted") curve shifted by 10% of a
    errs_mape.append(round(mape(vals1,vals2),4))
    errs_wape.append(round(wape(vals1,vals2),4))
    _ = ax0.plot(months, vals1, color="blue", marker='.')

_ = ax1.plot(levels, errs_mape, color="red" , marker='.')
_ = ax1.plot(levels, errs_wape, color="blue", marker='.')
_ = ax2.plot(levels, errs_wape, color="blue", marker='.')
_ = ax2.plot(levels, errs_wape, color="blue", marker='.')
_ = ax2.set(ylim=(0.0, myscale(np.max(errs_wape))))

fig.tight_layout()
plt.subplots_adjust(wspace=0.50, hspace=1.0)
plt.show() ;
```



In [8]:
```
mape_max = np.max(errs_mape)
mape_min = np.min(errs_mape)
wape_max = np.max(errs_wape)
wape_min = np.min(errs_wape)

print(f"MAPE max/min = {mape_max:9.4f} / {mape_min:6.4f} = {(mape_max/mape_
print(f"WAPE max/min = {wape_max:9.4f} / {wape_min:6.4f} = {(wape_max/wape_
```

```
MAPE max/min =    4.6138 / 0.0074 =    623.4865
WAPE max/min =    0.0139 / 0.0124 =      1.1210
```

In [9]:
```
print("WAPE:")
for v in errs_wape:
    print(f"{v:.4f}", end=",")
print()
print("MAPE:")
for v in errs_mape:
    print(f"{v:.4f}", end=",")
print()
```

```
WAPE:
0.0136,0.0132,0.0134,0.0139,0.0129,0.0136,0.0126,0.0134,0.0136,0.0129,0.0
130,0.0134,0.0139,0.0135,0.0135,0.0134,0.0131,0.0130,0.0124,0.0130,0.013
0,0.0139,0.0132,0.0137,0.0134,0.0137,0.0130,
MAPE:
0.0078,0.0103,0.0153,0.0200,0.0307,0.0415,0.0531,0.0969,4.6138,1.3665,0.3
920,0.4629,0.3518,0.4292,0.3648,0.6031,0.6717,0.5056,0.5353,0.0741,0.050
7,0.0385,0.0298,0.0188,0.0151,0.0103,0.0074,
```

So MAPE is really bad (can spike to very big numbers)

```
In the example above we were calculating MAPE and WAPE errors
between two sinusoidal curves shifted vertically by 0.2,
which is 10% of oscillation amplitude.
Also added some noise.

We repeated this calculation while raising both curves
together up or down on different levels.

When both curves are raised far from zero, then the
difference (0.2) between them is relatively small,
so both MAPE and WAPE are small (approx 0.01).

But when the arrays oscillate closer to zero,
the relative error of MAPE can spike to very big numbers.

So WAPE gives much more stable error estimates.

Also note that in this version of MAPE we used this logic:

    if abs(x) < 0.0001:
       x = 0.0001

which is much more stable than the original logic:
```

```
    if x == 0
       x = 0.0001

Original logic was causing even more instability
because x can be very small (for example, 0.00000000001)
and still not equal to zero. So this small value
will be used in the denominator causing huge MAPE values
```

How we made error estimates more stable

```
Consider different types of curves:
 type 1 - curve var up from zero (or whole curve is far below zero)
 type 2 - curve crosses zero or oscillates close to zero

Apparently that relative error will be different:
 type 1 - small relative error
 type 2 - big relative error

If we are estimating error for different segments
along the same curve (moving the window), how
can we compare errors of two segments where
one segment is far from zero (type 1),
and another segment is close to zero (type 2) ?

One possible solution may be to transform the data
to similar format. For example, if both segments
have similar up/down amplitudes, but differ in their
distance from zero, we may adjust these distances
to be the same (when calculating WAPE).
```

```
Suppose "vals" is an array of values in a segment.

    v_max = np.max(vals)
    v_min = np.min(vals)
    v_amp = v_max-v_min    # amplitude
    v_ave = np.mean(vals)  # average

Then we can shift values like this:

    vals_shifted = (vals - v_min) + 5 * v_amp

This shifted curve will oscillate in range
between 5 and 6 units of v_amp
```

```
But what about situations when in some segments
the v_amp is zero or too small ?

Consider these examples:
    v == 100   , v_amp == 0
    v == 0.0   , v_amp == 0
    v == small , v_amp = small

OK, what it means "small" ?

Let's define a metrics "v_scale" as following:
```

```
N = len(v)
v_scale = sum(abs(v)) / N

So for N points v = [100,100,..., 100]
the v_scale = 100, v_amp = 0

We probably not interested in variations less than 0.01% of the scale.

So for cases when v_amp <= 1e-4 * v_scale
we may presume that v == const

if (v_amp <= 1e-4 * v_scale):
    # use v_scale to make shift
    vals_corrected = (actual - v_min) + v_scale
else:
    # use v_amp to make shift
    vals_corrected = vals-v_min + 5*v_amp

Actual implementation above has more checks to avoid deviding by zero.
```

```
If we need to compare or average errors
in different segments of the same long time series,
we can use the same denominator for WAPE calculations.
We can pass it into wape() function.
```

In [ ]:

```

```